Debug and Run Diagnostics

Learning Objectives

After completing this unit, you'll be able to:

- Understand which debugging features are available on the Lightning Platform
- Use the Log Inspector in Developer Console to examine debug logs

Follow Along with Trail Together

Want to follow along with an expert as you work through this step? Take a look at this video, part of the Trail Together series.

(This clip starts at the 40:30 minute mark, in case you want to rewind and watch the beginning of the step again.)

Taking a Step Back

We're not going to try and tell you that debugging on the Lightning Platform is just as easy as it is in Visual Studio. To be perfectly honest, it's not even close. Debugging in a multi-tenant cloud environment presents unique challenges. That doesn't mean you can't do debugging and diagnostic work on the Lightning Platform. It's just that debugging is different from what you're used to. The good news is that in this area Salesforce has made enormous strides recently, and more is coming with every release.

```
System.debug('My Debug Message');
```

Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Debug and Run Diagnostics

You can also specify one of the following logging levels.

- NONE
- ERROR
- WARN
- INFO
- DEBUG
- FINE
- FINER
- FINEST

These levels run from lowest to highest and are cumulative. So if you pick the finest level, you get all messages that are logged as error, warn, info, and so on. There are also several debug log categories, and the amount of information logged depends on the log level. Learn more about debugging for different users by checking out the links in Resources.

And why is all this so important to know?

Ok, so we know you've heard a lot about limits, but understanding limits are critical to your success, so bear with us. Each debug log must be 20 MB or smaller. If it exceeds this amount, you won't see everything you need. Additionally, each org can retain up to 1,000 MB of debug logs. The oldest logs are overwritten.

Because debug logs are your primary way of getting debug information about your application, you want to make sure to not exceed these limits. If you never see an error message, you can't possibly do anything to address it. Check out the links in Resources about Advanced Apex Debugging and Best Practices for tips on how to avoid these kinds of issues.

Use the Log Inspector

Developer Console has grown quite a bit in the past few releases. One of its more useful features is the Log Inspector. To see how it works, let's walk through running some anonymous code and viewing the results.

1. From Setup ( ), select **Developer Console**.
2. In the Developer Console, select **Debug > Change Log Levels**.
3. Click the **Add/Change** link in General Trace Setting for You.
4. Select **INFO** as the debug level for all columns.
5. Click **Done**.
6. Click **Done**.
7. Select **Debug > Perspective Manager**.
8. Select **All (Predefined)** and click **Set Default**.
9. Click **Yes** to change this to your default perspective.
10. Close the Developer Console Perspective window.

Now let's execute some Apex.

1. Select **Debug > Open Execute Anonymous Window**.
2. Delete any existing code, and insert the following snippet:

```
System.debug(LoggingLevel.INFO, 'My Info Debug Message');
System.debug(LoggingLevel.FINE, 'My Fine Debug Message');
List<Account> accts = [SELECT Id, Name FROM Account];
for(Account a : accts) {
    System.debug('Account Name: ' + a.name);
```

And the results.

1. Select **Debug > Switch Perspective > All (Predefined)**.
2. Examine the results in the Timeline and Executed Units tabs.
3. Under Execution Log, select the **Filter** option, and enter **FINE**. Because we set the debug level to INFO for Apex Code, no results appear.
4. Select **Debug > Change Log Levels**.
5. Click the **Add/Change** link in General Trace Setting for You.
6. Change the DebugLevel for ApexCode and Profiling to **FINEST**.
7. Click **Done**.
8. Click **Done**.

Run the code again.

1. Select **Debug > Open Execute Anonymous Window**.
2. Leave the code that is currently there, and click **Execute**.
3. Under Execution Log, select the **Filter** option, and enter **FINE**. The filter search is case sensitive. You now see "My Fine Debug Message" displayed. You should also notice a size difference between the two latest logs in the Logs tab.

Set Checkpoints

As a .NET developer, you're used to setting breakpoints in your applications. But in a cloud-based, multi-tenanted environment where everyone is sharing resources, allowing everyone to halt execution and keep database connections open is disastrous.

Checkpoints are similar to breakpoints in that they reveal a lot of detailed execution information about a line of code. They just don't stop execution on that line.

To see how they work, let's walk through setting a checkpoint on a line of code that you created in an earlier unit on Understanding Execution Context. If you haven't created the handler and trigger code for the AccountTrigger, go back to that unit and complete that section before continuing.

1. In the Developer Console, select **File > Open**.
2. Select **Classes** as the entity type, and **AccountHandler** as the entity.
3. Click **Open**.
4. Position your cursor over line 10 in the left margin and click once. A red dot appears next to the line number.
5. Double-click the latest entry in the Logs panel to open the debug log.
6. Select **Debug > Open Execute Anonymous Window**.
7. Delete any existing code, and insert the following snippet:

```
Account acct = new Account(
    Name='Test Account 3',
    Phone='(415)555-8989',
    NumberOfEmployees=30,
    BillingCity='San Francisco');
insert acct;
```

8. Make sure that **Open Log** is selected, and click **Execute**.
9. Click the **Checkpoints** tab, and double-click the first entry that appears. The Checkpoint Inspector appears.
10. On the Symbols tab, expand the nodes within the execution tree. Notice the Key and Value columns.
11. Click the **Heap** tab. Notice the Count and Total Size columns.

Resources

Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Debug and Run Diagnostics

- *Apex Developer Guide*: Debug Log (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_debugging_debug_log.htm)
- *Apex Developer Guide*: Working with Logs in the Developer Console (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_debugging_system_log_console.htm)
- *Salesforce Help*: Examples of Using the Log Inspector (https://help.salesforce.com/s/articleView?id=platform.code_dev_console_solving_problems_using_system_log.htm&type=5)
- *Salesforce Help*: Set Checkpoints in Apex Code (https://help.salesforce.com/s/articleView?id=platform.code_dev_console_checkpoints_setting.htm&type=5)

Quiz

+100 Points

To complete this unit, you need to answer all the quiz questions correctly.

1 **The primary source of debug information on the Lightning platform is the**

A Apex Verifier

B Apex Error Reporting Log

C Debug Log

D Stack Log

2 **Which of the following is true regarding checkpoints?**

A Execution is available only with production orgs.

B You cannot use checkpoints with Asynchronous Apex.

C You cannot use checkpoints with triggers.

D Execution doesn't stop on the line with the checkpoint.

Apex & .NET Basics (/content/learn/modules/apex_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Debug and Run Diagnostics